BUILDER SKILLS · PART 15

Stop Calling OpenAI From 47 Places.

Every serious AI product ends up with one internal endpoint in front of every model provider. That endpoint is the LLM gateway. Here is what it does and how to build or buy one.



15 slides · responsibilities · routing config · fallbacks · build vs buy · pitfalls

AI Coach John Follow

PROITBRIDGE
Strive For Better Future

Repost

What an LLM gateway actually is

A reverse proxy that speaks one API to your services and many APIs to providers. Every cross-cutting concern lives there once instead of in every app.



Without it

Provider keys in 12 repos, four retry implementations, nobody knows which team spent $9K last week, and a model deprecation means 12 pull requests.

With it

Apps hold a gateway key. Model names are aliases. Swapping providers, adding a fallback or capping a team's spend is a config change.

Most teams adopt the OpenAI chat-completions schema as the internal contract because every SDK and framework already speaks it.

Nine jobs, one place

These are the concerns that otherwise get copy-pasted, half-finished, into every service that calls a model.

1. **Auth & virtual keys.** Per team, per app, revocable, never the provider key.
2. **Routing & aliases.** *fast-cheap* → gpt-4.1-mini today, haiku tomorrow.
3. **Fallback & retries.** 429/5xx → backoff, then another provider.
4. **Rate limits & quotas.** RPM and TPM per key, monthly budgets per team.
5. **Cost attribution.** Tokens × price, tagged by team, feature, customer.
6. **PII redaction.** Mask emails, cards, IDs before they leave your network.
7. **Logging & tracing.** Prompt, response, latency, model, trace id. Sampled or full.
8. **Response caching.** Exact-match or semantic, for repeated queries.
9. **Schema normalisation.** One request/response shape across every provider.

\+ **Often added later:** guardrails, prompt registry, A/B routing, streaming passthrough.

Rule of thumb: if you would write the same middleware in a second service, it belongs in the gateway.

What happens to one request

Order matters. Cheap checks run first so a rejected request never costs a token.



steps 1–4 cost microseconds and reject most bad traffic · step 7 is where the money and the latency go

Latency budget

A good gateway adds ≈5–20 ms. If yours adds 200 ms, the redaction or logging step is synchronous and needs to move off the hot path.

Streaming

Pass SSE chunks through untouched, then meter and log when the stream closes. Never buffer the whole response to count tokens.

Model aliases: never hardcode a model

Apps ask for a capability tier. The gateway decides which model, provider and region fulfils it today.

ALIAS APPS USE	RESOLVES TO (THIS QUARTER)	FALLBACK CHAIN
fast-cheap	`gpt-4.1-mini`	`claude-haiku-4-5 → gemini-2.5-flash`
balanced	`claude-sonnet-5`	`gpt-4.1 → bedrock/sonnet`
frontier	`claude-opus-5-5`	`gpt-5 → none (fail loud)`
embed	`text-embedding-3-large`	`none (vectors must match)`
internal-pii	`vllm/llama-3.3-70b`	`none (data may not leave VPC)`

ROUTING STRATEGIES THE ALIAS CAN HIDE

Weighted
90% current, 10% candidate. Compare quality and cost live.

Latency-aware
Pick the deployment with the lowest rolling p95 across regions.

Cost-aware
Short prompts → cheap model, long context → the one with caching.

⚠ **Embeddings and fine-tuned models must not fail over. Different embedding model, different vectors, broken index.**

Retries, timeouts, fallbacks

Providers throw 429s and 529s every week. The gateway turns those into a slightly slower answer instead of an outage.



total worst case ≈ timeout + 2 s of backoff + fallback latency · set the client deadline accordingly

circuit breaker: after 5 failures in 30 s, skip the primary for 60 s instead of paying the timeout every call

SIGNAL	RETRY?	FALL OVER?	NOTE
429 rate limit	Yes, with backoff	Yes	Honour Retry-After if present
5xx / 529 overloaded	Once	Yes	Provider-side, not your fault
Timeout	Careful	Yes	Streaming: use time-to-first-token, not total
400 bad request	No	No	Your bug, retrying wastes money
Content filter	No	Maybe	Another provider may accept it, decide per policy

Retries need **idempotency keys** for tool calls and writes. A retried "send the refund" request must not refund twice.


AI Coach John Follow PROITBRIDGE Strive For Better Future Repost

Routing and fallbacks as config

LiteLLM proxy syntax, the most common open-source gateway. Portkey, Kong and Cloudflare express the same ideas their own way.

```yaml
# litellm_config.yaml
model_list:
  - model_name: balanced              # the alias apps call
    litellm_params:
      model: anthropic/claude-sonnet-5
      api_key: os.environ/ANTHROPIC_KEY
      timeout: 30
      rpm: 600
  - model_name: balanced              # same alias → load-balanced
    litellm_params:
      model: bedrock/anthropic.claude-sonnet-5
      aws_region_name: ap-south-1

router_settings:
  routing_strategy: latency-based-routing
  num_retries: 2
  fallbacks: [{balanced: [openai/gpt-4.1]}]
  allowed_fails: 5          # then cooldown 60 s
```

</> Apps then call **base_url=https://llm.internal/v1** with any OpenAI SDK and **model="balanced"**.

Send **metadata: {team, feature, customer_id}** on each call. That is what makes cost reports possible.

Virtual keys, quotas and the bill

The gateway is the only place that sees every token. That makes it the natural source of truth for who spent what.



MONTHLY REPORT · support $6,400 · search $2,100 · evals $900 (batched) · agents $4,300 ▲ 210% ← investigate
rotate a virtual key in seconds · the provider key never leaves the gateway host

CONTROL	GRANULARITY	EXAMPLE
Budget	Key, team, customer	Free-tier customers capped at $2 / month, hard stop
Rate limit	RPM, TPM per key	Evals key: 50 RPM so it never starves the product
Model allow-list	Per key	Interns' key can call fast-cheap only
Max tokens	Per route	Chat route caps output at 1,500 to bound cost

Cost per feature is the metric leadership actually asks for. Without gateway metadata you are reverse-engineering it from provider invoices.

Redaction and tracing

Two jobs that must happen on every call, and that nobody adds to the fifth service.

Redact before egress

Regex plus a small NER model (Presidio, GLiNER) swaps emails, phone numbers, card and Aadhaar-style IDs for placeholders. Re-hydrate placeholders in the response if the app needs them.

Trace every hop

Emit OpenTelemetry spans: gateway → provider, with model, tokens, cost, cache status, retry count. Export to Langfuse, Arize, Datadog or your own store.

WHAT A GOOD LOG LINE CARRIES

`trace_id` `team` `feature` `alias` `resolved_model` `provider` `region` `in_tok` `out_tok` `cached_tok` `cost_usd` `ttft_ms` `total_ms` `retries` `fallback_used` `status`

STORE	WHAT	RETENTION
Traces	Every request's metadata, no prompt bodies	30–90 days
Payloads	Prompt and response text, sampled 1–5% or on error	7–30 days, access-controlled

Log payloads *after* redaction, never before. Otherwise the gateway becomes the biggest PII store in the company.

Canary a new model behind the alias

Model upgrades are deploys. The gateway lets you roll them out like any other deploy: a slice of traffic, metrics, then promote or roll back.



day 1–3: 5% · judge scores + thumbs-down rate

day 4–7: 25% → 50% → 100% if no regression

rollback = one config line · no app redeploy · the reason the alias exists

✓ Promote when

Judge score within noise or better, cost per call lower or justified, p95 latency not worse by more than your SLO allows, error rate flat.

✕ Roll back when

Tool-call format errors rise, refusal rate jumps, JSON schema failures appear. New models break structured output more often than they break prose.

Shadow mode is the safer first step: send the candidate the same request, discard its answer, log both. Zero user impact, real traffic.

Build vs buy: the landscape

Five realistic choices (Helicone is a sixth, observability-first). Pick by VPC needs and how much you want to operate.

OPTION	TYPE	STRENGTH	TRADE-OFF
LiteLLM proxy	OSS, self-host	100+ providers, OpenAI schema, virtual keys, spend DB	You run Postgres + Redis, Python perf ceiling
Portkey	SaaS or hybrid	Routing configs, guardrails, prompt registry, observability	Vendor in the hot path unless hybrid
Kong AI Gateway	OSS + enterprise	Reuses your existing API gateway, plugins, Lua/Go perf	Heavier to set up, fewer LLM-native features
Cloudflare AI GW	SaaS, edge	Caching, rate limits, logs at the edge, near-zero ops	Data transits Cloudflare, limited custom logic
Custom FastAPI	Build	Exactly your rules, your schema, your VPC	You own retries, streaming, metering forever

Build when

Regulated data, unusual providers (internal vLLM fleets), or you already have a platform team and an API gateway.

Buy when

Under ≈10 engineers, multiple SaaS providers, and you want cost dashboards this week, not this quarter.

HANDS ON

The 25-line FastAPI skeleton

If you build, start here. Aliases, fallback and metering in one file, then grow it. Streaming and redaction come next.

```python
from fastapi import FastAPI, Header, HTTPException
import litellm

app = FastAPI()
ALIASES = {"balanced": ["anthropic/claude-sonnet-5", "openai/gpt-4.1"]}

@app.post("/v1/chat/completions")
async def chat(body: dict, authorization: str = Header()):
    team = resolve_virtual_key(authorization)        # DB lookup, cached
    check_budget_and_rpm(team)                       # raise 429 early
    body["messages"] = redact_pii(body["messages"])
    for model in ALIASES[body["model"]]:             # fallback loop
        try:
            r = await litellm.acompletion(**{**body, "model": model},
                                          timeout=30, num_retries=2)
            meter(team, model, r.usage)                 # cost + trace
            return r
        except (litellm.RateLimitError, litellm.ServiceUnavailableError):
            continue
    raise HTTPException(503, "all providers unavailable")
```

litellm as a library gives you provider translation for free. The gateway logic around it is what you own.



Where gateways go wrong

A gateway concentrates risk as well as control. Each of these has taken down a production AI product.

1. **Single point of failure.** One pod, one region, one Redis. Run at least two replicas behind a load balancer and make the gateway stateless where possible.
2. **Hidden latency.** Synchronous logging to Postgres on every call adds 50–150 ms. Log through a queue.
3. **Fallback that changes behaviour.** Falling from Sonnet to a mini model silently breaks a JSON schema the app relies on. Fall over within a capability tier only.
4. **Lost provider features.** Normalising to one schema drops prompt caching flags, extended thinking, or vision blocks. Pass provider extras through.
5. **Streaming through a buffer.** Time-to-first-token goes from 400 ms to 6 s because the gateway waits for the full response to count tokens.
6. **Hidden costs.** SaaS gateways price per request or per token. At 50M calls a month, the gateway can cost more than the smallest model you run.

Give the gateway its own SLO and dashboard. When "the AI is slow", the first question is: gateway or provider?

Gateway launch checklist

Ship it in this order. Each step is useful on its own, and nothing later depends on skipping something earlier.

1. One internal URL, OpenAI-compatible schema, provider keys in a vault.
2. Virtual keys per team and app. Delete every provider key from app repos.
3. Aliases for 3–4 capability tiers. Ban raw model names in app code via lint.
4. Retries with jitter, per-model timeouts, fallback within the same tier.
5. Metadata on every call: team, feature, customer. Cost dashboard by week 2.
6. Budgets and RPM per key, with alerts at 80%.
7. PII redaction on egress, payload logging sampled and post-redaction.
8. Two replicas, health checks, its own SLO, canary routing for model upgrades.

≈5–20	2+	0	1 line
ms added latency, healthy	replicas, always	provider keys outside the gateway	to swap or roll back a model

The LLM gateway on one slide

CONCEPT	REMEMBER
Gateway	One internal endpoint in front of all providers. Cross-cutting concerns live there once.
Contract	OpenAI chat-completions schema internally, translated per provider.
Virtual keys	Per team and app, revocable, budgeted. Provider keys never leave the gateway.
Aliases	fast-cheap, balanced, frontier, embed. Apps never name a real model.
Resilience	Retry 429/5xx with jitter, per-model timeouts, fallback inside the same tier, circuit breaker.
Idempotency	Keys on retried requests so tool calls and writes never run twice.
Governance	Budgets, RPM/TPM, model allow-lists, cost by team/feature/customer.
Data	Redact PII on egress. Log metadata always, payloads sampled and post-redaction.
Rollouts	Shadow, then 5% → 25% → 100% behind the alias. Rollback is a config line.
Buy	LiteLLM, Portkey, Kong AI, Cloudflare AI Gateway, Helicone.
Pitfalls	SPOF, sync logging latency, cross-tier fallbacks, buffered streams, lost provider features.

Every model call goes through one door. Keys, routing, retries and the bill all live behind it.


AI Coach John Follow
PROITBRIDGE Strive For Better Future
Repost